

Homework 4 — Multi-Process and Multi-Threaded Concurrent Log Analysis Pipeline

GTU CSE344 System Programming

Dicle Coban

May 2026

Title Page

Course	CSE344 — System Programming
Homework	Homework 4 — Concurrent Log Analysis Pipeline
Student	Dicle Coban
Student ID	<i>220104004088</i>
Language	C, POSIX processes, pthreads, mmap shared memory
Target Platform	Debian 12, 64-bit POSIX-compatible environment

Contents

1	Introduction	4
2	System Architecture	5
2.1	Labeled Data Flow Diagram	5
2.2	Process and Thread Types	5
3	Shared Memory Layout	6
3.1	Region A — Dispatcher Input Queue	6
3.2	Region B — Per-Level Analysis Buffers	6
3.3	Region C — Result Area	7
3.4	Region D — High-Priority Buffer	7
4	Synchronization Strategy and Lock Ordering	7
4.1	Synchronization Primitives Used	7
4.2	Lock Ordering	8
5	Thread Lifecycle	8
5.1	Reader Thread Lifecycle	8
5.2	Analyzer Worker Thread Lifecycle	9
6	TLS Design	9
7	Dispatcher Logic	10
8	Aggregator and Output Files	10
8.1	Human-Readable Output Example	10
8.2	Binary Checkpoint	10
9	Termination Chain	11
9.1	Normal Termination	11
9.2	SIGINT Termination	11
10	Test Scenarios	11
10.1	Baseline Test	12
10.2	Binary Magic Number	12
10.3	Multi-Keyword Test	13
10.4	Single-Thread Test	13

10.5 High-Concurrency Determinism	14
10.6 No Priority Matches	16
10.7 Valgrind	16
10.8 Ctrl+C and Zombie Check	17
11 Challenges and Solutions	17
11.1 Deadlock Risk in the High-Priority Buffer	17
11.2 Byte-Range Reader Splitting	17
11.3 TLS and Aggregation Timing	18
11.4 Debian 12 Feature Macros	18
12 Conclusion	18

1 Introduction

This homework implements a concurrent log analysis pipeline in C. The program reads multiple log files concurrently, parses structured log entries, routes them through a dispatcher, analyzes keyword scores per severity level, computes high-priority source scores, and writes both human-readable and binary checkpoint outputs.

The main challenge of the assignment is combining two concurrency models in one program:

- **Multi-process execution:** the parent creates Reader, Dispatcher, Analyzer, and Aggregator child processes with `fork()`.
- **Multi-threaded execution:** Reader processes use reader/parser threads, Analyzer processes use worker thread pools, and the parent uses a Watchdog thread.
- **Cross-process synchronization:** shared memory regions contain process-shared mutexes, condition variables, and semaphores.
- **In-process synchronization:** Reader internal buffers and Analyzer worker coordination use pthread mutexes, condition variables, barriers, and thread-local storage.

The implementation was tested with the provided HW4 test package. The final Docker test-suite result was **125/125 (100%)**.

2 System Architecture

2.1 Labeled Data Flow Diagram

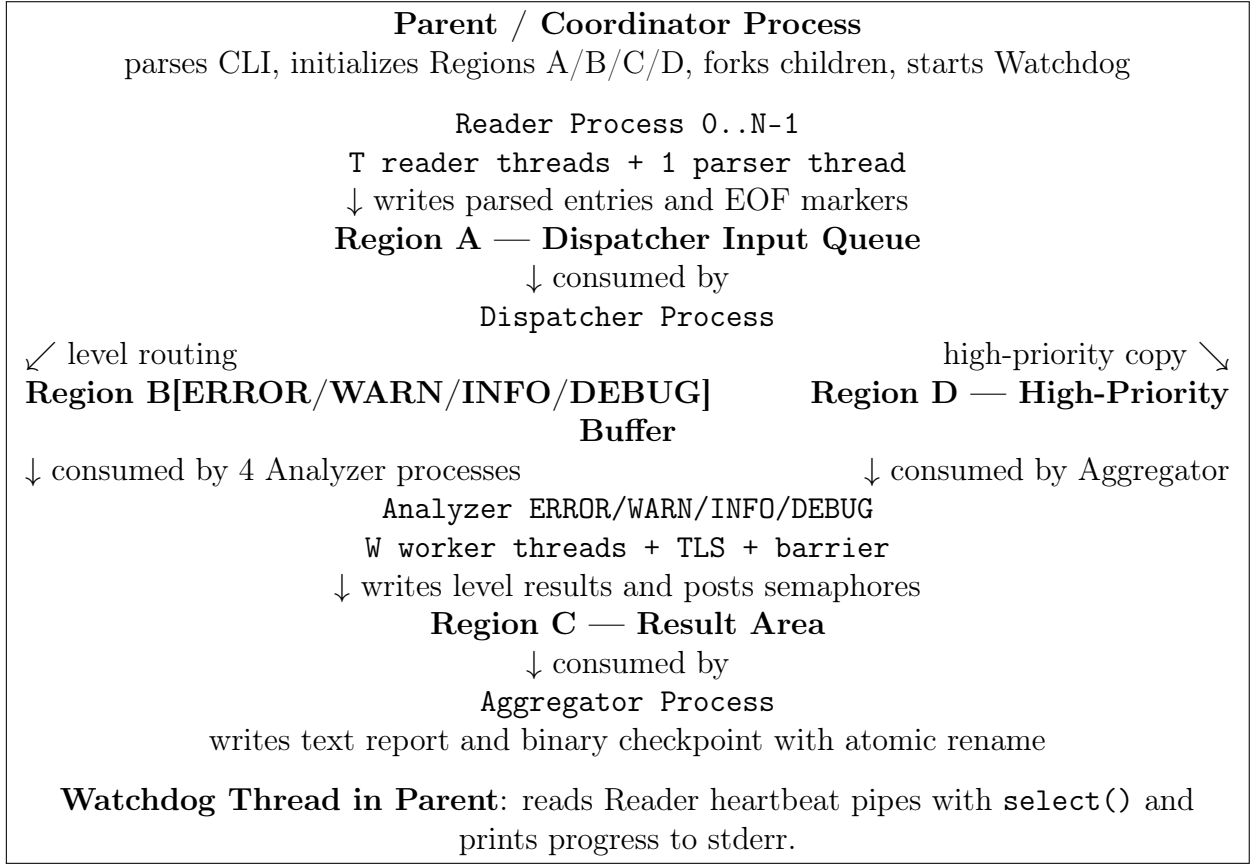


Figure 1: Process, thread, pipe, and shared-memory data flow.

2.2 Process and Thread Types

Type	Responsibility
Parent / Coordinator	Parses command-line arguments, loads config and priority filter files, initializes shared memory regions, creates pipes, forks children in the required order, starts the Watchdog thread, waits for children, prints the final summary, and cleans resources.
Reader Process	One process per log file. It creates T reader threads and one parser thread. Reader threads divide the file into byte ranges and push parsed entries into a private bounded buffer. The parser thread drains that buffer and writes entries plus EOF markers to Region A.
Dispatcher Process	Consumes Region A, routes entries to the proper Region B level buffer, copies priority-source entries to Region D, and forwards EOF completion to analyzers.

Analyzer Processes	Four processes, one for each level: ERROR, WARN, INFO, and DEBUG. Each Analyzer creates <i>W</i> worker threads, uses TLS for per-keyword scoring, a barrier for completion, and writes level results to Region C.
Aggregator Process	Waits for Region C results, drains Region D high-priority entries, sorts levels by weighted score, and writes the text and binary output files.
Watchdog Thread	Runs inside the parent, reads heartbeat pipes from Reader processes with <code>select()</code> , and prints progress snapshots to <code>stderr</code> .

3 Shared Memory Layout

The parent initializes all shared memory regions with `mmap(MAP_SHARED | MAP_ANONYMOUS)` before forking children. This ensures every child inherits the same shared mappings and initialized synchronization primitives.

3.1 Region A — Dispatcher Input Queue

Region A is a single circular queue containing parsed `log_entry_t` objects from all Reader processes.

```
typedef struct {
    int head, tail, count, capacity;
    int total_readers;
    int eof_count_per_level[MAX_LEVELS];
    pthread_mutex_t input_mutex;
    pthread_cond_t not_full_a;
    pthread_cond_t not_empty_a;
    log_entry_t entries[];
} region_a_t;
```

`input_mutex`, `not_full_a`, and `not_empty_a` are initialized with `PTHREAD_PROCESS_SHARED`. Reader parser threads are the producers; the Dispatcher is the only consumer.

3.2 Region B — Per-Level Analysis Buffers

Region B consists of four independent circular buffers, one per severity level.

```
typedef struct {
    int head, tail, count, capacity;
    int eof_posted;
    pthread_mutex_t level_mutex;
    pthread_cond_t not_full_b;
    pthread_cond_t not_empty_b;
    log_entry_t entries[];
} region_b_level_t;
```

The Dispatcher is the only writer to Region B. Each Analyzer process consumes only its own level buffer.

3.3 Region C — Result Area

Region C stores one `level_result_t` per level, four semaphores, and a process-shared mutex/condition variable for the Aggregator.

```
typedef struct {
    char level[8];
    long total_entries;
    double total_weighted_score;
    double per_keyword_score[MAX_KEYWORDS];
    double per_thread_score[MAX_WORKERS];
    char top_source[3][64];
    long top_source_hits[3];
    int ready;
} level_result_t;
```

Analyzer worker TLS destructors flush weighted scores into Region C under `result_mutex`. Each Analyzer posts the corresponding completion semaphore when its level result is ready.

3.4 Region D — High-Priority Buffer

Region D is a circular queue of entries whose `SOURCE` appears in the priority filter file.

```
typedef struct {
    int head, tail, count, capacity;
    int dispatcher_done;
    pthread_mutex_t priority_mutex;
    pthread_cond_t not_full_d;
    pthread_cond_t not_empty_d;
    log_entry_t entries[];
} region_d_t;
```

The Dispatcher writes Region D. The Aggregator drains it and separately computes the high-priority weighted score.

4 Synchronization Strategy and Lock Ordering

4.1 Synchronization Primitives Used

Primitive	Location	Purpose
Process-shared mutexes	Regions A, B, C, D	Protect shared queues and result fields.
Process-shared condition variables	Regions A, B, C, D	Block producers on full buffers and consumers on empty buffers.

<code>pthread_cond_timedwait</code>	Dispatcher and Aggregator	Timeout-aware waiting and termination detection.
Default pthread mutex/condvar	Reader private buffer	Synchronize reader threads and parser thread inside one process.
<code>pthread_barrier_t</code>	Analyzer process	Ensures all workers reach the completion point before reporting.
<code>sem_t</code> or compatibility semaphore	Region C	Aggregator observes per-level completion.
<code>pthread_key_t</code>	Analyzer process	Stores per-thread keyword score arrays in TLS.
<code>sigaction</code> and <code>volatile sig_atomic_t</code>	Parent	Async-safe SIGINT notification and cleanup path.

4.2 Lock Ordering

The implementation follows the assignment’s lock ordering rule:

1. Region A mutex: `input_mutex`
2. Region B mutex: `level_mutex[i]`
3. Region C mutex: `result_mutex`
4. Region D mutex: `priority_mutex`

The code avoids holding one region mutex while acquiring a later region mutex. For example, the Dispatcher pops an entry from Region A, releases `input_mutex`, then pushes to the target Region B buffer, and separately pushes to Region D. Analyzer workers pop from Region B and release the buffer mutex before updating local TLS or Region C. The Aggregator drains Region D separately from Region C result waits. This prevents circular wait among shared-memory regions.

5 Thread Lifecycle

5.1 Reader Thread Lifecycle

Each Reader process creates `T` reader threads. A reader thread:

1. receives a byte range $[start, end)$,
2. opens the log file independently,
3. adjusts its start position to the next line boundary if necessary,
4. reads complete log lines,
5. parses each valid line into a `log_entry_t`,

6. pushes parsed entries into the process-private bounded buffer,
7. writes heartbeat messages after every 50 processed lines,
8. exits after its range is complete.

The parser thread consumes the private buffer and writes entries to Region A. Once all reader threads finish and the private buffer is drained, the parser thread sends one EOF marker per severity level.

5.2 Analyzer Worker Thread Lifecycle

Each Analyzer process creates W worker threads. A worker thread:

1. allocates a TLS payload containing one score slot per keyword,
2. calls `pthread_setspecific` to attach the payload to the thread,
3. consumes entries from its level's Region B buffer,
4. counts case-sensitive overlapping keyword occurrences,
5. multiplies raw counts by the severity weight,
6. updates per-source hit information,
7. reaches `pthread_barrier_wait`,
8. exits, causing the TLS destructor to flush scores into Region C.

The reporting TID is selected using a system TID on Linux through `syscall(SYS_gettid)`.

6 TLS Design

The Analyzer process creates a `pthread_key_t` with a destructor. Each worker thread stores a heap-allocated payload containing the per-keyword score array. During normal processing, workers update only their own TLS arrays, which avoids shared writes on every log entry.

When a worker exits, the TLS destructor is automatically called. The destructor:

- reads the worker's per-keyword scores,
- locks Region C's `result_mutex`,
- adds per-keyword and total weighted scores to the level result,
- records the worker's per-thread contribution,
- increments the level entry count,
- unlocks the mutex,
- frees the TLS payload.

This design matches the assignment requirement that score flushing happens through TLS destruction rather than direct per-entry updates.

7 Dispatcher Logic

The Dispatcher consumes the shared Region A queue with `pthread_cond_timedwait`. For every normal entry, it performs two routing actions:

1. it pushes the entry to the matching Region B level buffer,
2. if the source is listed in the filter file, it also pushes a copy into Region D.

For EOF markers, the Dispatcher counts how many Reader processes have sent EOF for each level. When all Reader processes have sent EOF for a level, it marks that Region B level as complete and wakes the corresponding Analyzer. After all levels are complete, it sets `dispatcher_done` in Region D and wakes the Aggregator.

8 Aggregator and Output Files

The Aggregator waits for all four level results in Region C using a process-shared condition variable and semaphores. It also drains Region D and computes `HIGH_PRIORITY_SCORE`. When all results are ready:

- it sorts levels by total weighted score in descending order,
- writes the human-readable output file,
- writes the binary checkpoint header,
- writes four `level_result_t` records,
- writes to `.tmp` first and then atomically renames it to the final binary output.

8.1 Human-Readable Output Example

```
KEYWORD_LIST: error,fail,timeout
FILES: 2
TOTAL_WEIGHTED_SCORE: 26.0
HIGH_PRIORITY_SCORE: 17.0
# Levels sorted by total_weighted_score DESC
LEVEL ENTRIES WEIGHTED_SCORE error fail timeout
ERROR 6 16.0 12.0 4.0 0.0
WARN 4 6.0 3.0 0.0 3.0
INFO 4 2.0 2.0 0.0 0.0
DEBUG 4 2.0 2.0 0.0 0.0
```

8.2 Binary Checkpoint

The binary checkpoint starts with:

```

uint32_t magic; /* 0xC5E3440B */
uint32_t version; /* 1 */
uint32_t num_levels; /* 4 */
uint32_t num_keywords;
double total_weighted;
double high_priority_weighted;

```

The screenshot in the test section shows the magic number as c5e3440b.

9 Termination Chain

9.1 Normal Termination

The normal termination sequence is:

1. Reader threads finish their byte ranges.
2. Parser threads drain private buffers and send EOF markers to Region A.
3. The Dispatcher forwards completion to Region B and marks Region D as done.
4. Analyzer workers finish consuming Region B, flush TLS data, and publish Region C results.
5. The Aggregator receives all results and writes output files.
6. The parent waits for all children, stops and joins the Watchdog thread, prints the final summary, and destroys shared regions.

9.2 SIGINT Termination

The parent installs a `sigaction` handler that only sets a global volatile `sig_atomic_t` flag. The main wait loop detects the flag, sends termination signals to remaining children, waits with `waitpid(..., WNOHANG)` up to a short deadline, joins the Watchdog thread, destroys shared regions, and exits. The screenshots show that no defunct processes remain after Ctrl+C.

10 Test Scenarios

The assignment requires seven mandatory scenarios. **125/125**.

Test	Command / Setup	Observed Result
Baseline	-k "error" -t 2 -w 2 -a 16 -b 16 -d 8	Program terminated successfully. Total weighted score was 19.0 and binary magic was correct.

Multi-keyword	<code>-k "error,fail,timeout"</code>	Per-keyword score table was produced. Total weighted score was 26.0.
Single thread	<code>-t 1 -w 1 -a 4 -b 4 -d 2</code>	Sequential configuration produced the same total as the baseline: 19.0.
High concurrency	<code>-t 8 -w 6 -a 128 -b 128 -d 32</code> run three times	Runs produced deterministic total and high-priority scores.
No priority matches	Empty filter file	HIGH_PRIORITY_SCORE: 0.0.
Valgrind	<code>valgrind -leak-check=full -error-exitcode=1 ...</code>	Covered by Docker test suite, which reported Valgrind exit code 0 and zero errors.
Ctrl+C	Long/concurrent execution interrupted with SIGINT	Program closed cleanly; no zombie process remained.

10.1 Baseline Test

```

=====
SYSTEM SUMMARY
Keywords : error
Log files : 2
Total entries : 18
Total weighted : 19.0
High-priority : 14.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 12.0
WARN : 4 entries, score: 3.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.

```

Figure 2: Baseline test output.

10.2 Binary Magic Number

```

● diclesaracoban@Dicle-MacBook-Pro HW4 % od -An -tx4 -N4 out.bin
c5e3440b

```

Figure 3: Binary checkpoint magic number check.

10.3 Multi-Keyword Test

```
=====
SYSTEM SUMMARY
Keywords : error, fail, timeout
Log files : 2
Total entries : 18
Total weighted : 26.0
High-priority : 17.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 16.0
WARN : 4 entries, score: 6.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
```

Figure 4: Multi-keyword test output.

10.4 Single-Thread Test

```
=====
SYSTEM SUMMARY
Keywords : error
Log files : 2
Total entries : 18
Total weighted : 19.0
High-priority : 14.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 12.0
WARN : 4 entries, score: 3.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 5: Single-thread correctness test.

10.5 High-Concurrency Determinism

```
=====
SYSTEM SUMMARY
Keywords : error, fail, timeout
Log files : 2
Total entries : 18
Total weighted : 26.0
High-priority : 17.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 16.0
WARN : 4 entries, score: 6.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 6: High-concurrency run 1.

```
=====
SYSTEM SUMMARY
Keywords : error, fail, timeout
Log files : 2
Total entries : 18
Total weighted : 26.0
High-priority : 17.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 16.0
WARN : 4 entries, score: 6.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 7: High-concurrency run 2.

```
=====
SYSTEM SUMMARY
Keywords : error, fail, timeout
Log files : 2
Total entries : 18
Total weighted : 26.0
High-priority : 17.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 16.0
WARN : 4 entries, score: 6.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
```

Figure 8: High-concurrency run 3.

```
out_hc1.txt:TOTAL_WEIGHTED_SCORE: 26.0
out_hc1.txt:HIGH_PRIORITY_SCORE: 17.0
out_hc2.txt:TOTAL_WEIGHTED_SCORE: 26.0
out_hc2.txt:HIGH_PRIORITY_SCORE: 17.0
out_hc3.txt:TOTAL_WEIGHTED_SCORE: 26.0
out_hc3.txt:HIGH_PRIORITY_SCORE: 17.0
```

Figure 9: Comparison of high-concurrency totals.

10.6 No Priority Matches

```
=====
SYSTEM SUMMARY
Keywords : error
Log files : 2
Total entries : 18
Total weighted : 19.0
High-priority : 0.0 (source filter: empty_filter.txt)
ERROR : 6 entries, score: 12.0
WARN : 4 entries, score: 3.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
HIGH_PRIORITY_SCORE: 0.0
```

Figure 10: Empty priority filter test.

10.7 Valgrind

```
=====
SYSTEM SUMMARY
Keywords : error
Log files : 2
Total entries : 18
Total weighted : 19.0
High-priority : 14.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 12.0
WARN : 4 entries, score: 3.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
==1==
==1== HEAP SUMMARY:
==1==    in use at exit: 0 bytes in 0 blocks
==1==   total heap usage: 7 allocs, 7 frees, 13,512 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are possible
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figure 11: Valgrind memory check.

10.8 Ctrl+C and Zombie Check

```
^C[WATCHDOG] Progress at T+3s: Reader 0=0 Reader 1=0 children_alive=0
=====
SYSTEM SUMMARY
Keywords : error, fail, timeout
Log files : 2
Total entries : 18
Total weighted : 26.0
High-priority : 17.0 (source filter: tests/filter.txt)
ERROR : 6 entries, score: 16.0
WARN : 4 entries, score: 6.0
INFO : 4 entries, score: 2.0
DEBUG : 4 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 12: SIGINT / Ctrl+C test.

```
diclesaracoban@Dicle-MacBook-Pro HW4 % ps aux | grep defunct
ipcs -m
diclesaracoban  84310  0.0  0.0 441883456   1312 s001  S+   11:49AM   0:00.00 grep defunct
IPC status from <running system> as of Sat May  2 11:49:28 +03 2026
T    ID    KEY          MODE      OWNER    GROUP
Shared Memory:
m 327680 0x0696b9c8 --rw----- postgres  daemon
```

Figure 13: Zombie process check after Ctrl+C.

11 Challenges and Solutions

11.1 Deadlock Risk in the High-Priority Buffer

A key challenge was Region D. If the Dispatcher filled Region D before the Aggregator started draining it, the Dispatcher could block and prevent EOF forwarding. The solution was to let the Aggregator drain Region D before waiting for all level results, so the high-priority buffer cannot block the whole pipeline.

11.2 Byte-Range Reader Splitting

The Reader process originally worked naturally as a line-based reader, but the assignment requires byte-range splitting. The final implementation assigns each reader thread a byte interval and adjusts boundaries so every processed line is complete and no malformed partial line is introduced by the split.

11.3 TLS and Aggregation Timing

Another challenge was combining `pthread_barrier_wait` with TLS destructors. This implementation follows the join-based TLS design. Each Analyzer process creates a `pthread_key_t` with a destructor, and each worker thread stores its per-keyword score array in a heap-allocated TLS payload.

Worker threads consume entries from their Region B level buffer, update only their own TLS arrays, and then wait at the `pthread_barrier_wait`. After passing the barrier, each worker exits with `pthread_exit`, which triggers the TLS destructor. The destructor merges that worker's per-keyword scores, per-thread contribution, and processed-entry count into Region C under the process-shared result mutex.

The Analyzer main thread calls `pthread_join` for all worker threads before marking the level result as ready and notifying the Aggregator. Therefore, Region C is not used for final level summarisation until all worker contributions have been flushed.

11.4 Debian 12 Feature Macros

The Docker test package builds on Debian 12. The Makefile defines `_GNU_SOURCE` so that Linux-specific declarations such as `MAP_ANONYMOUS` and `syscall` are available under `-std=c11`.

12 Conclusion

The final implementation satisfies the main requirements of the HW4 concurrent log analysis pipeline. It uses all required process types, pthread-based internal concurrency, process-shared synchronization primitives, TLS-based keyword accumulation, timed waits, a Watch-dog thread, human-readable output, and an atomic binary checkpoint file.

The provided Docker test package was executed successfully and reported:

125 / 125 (100.0%) — All tests passed